

**Московский авиационный институт
(национальный исследовательский университет)**

**Факультет информационных технологий и прикладной
математики**

Кафедра вычислительной математики и программирования

Лабораторная работа №1 по курсу «Дискретный анализ»

Студент: М. А. Хорохорин
Преподаватель: А. А. Кухтичев
Группа: М8О-206БВ-24
Дата: 05.03.2026
Оценка:
Подпись:

Москва, 2026

Лабораторная работа №1

Задача: Требуется разработать программу, осуществляющую ввод пар «ключ-значение», их упорядочивание по возрастанию ключа указанным алгоритмом сортировки за линейное время и вывод отсортированной последовательности.

Вариант сортировки: Поразрядная сортировка.

Вариант ключа: даты в формате DD.MM.YYYY, например 1.1.1, 01.09.2009, 31.12.2009.

Вариант значения: строки фиксированной длины 64 символа. Входные строки могут быть короче и дополняются нулями до 64 символов, которые не выводятся.

1 Описание

Требуется написать реализацию алгоритма поразрядной сортировки для пар «дата-значение».

Основная идея поразрядной сортировки заключается в том, чтобы каждый входной элемент разбить на разряды и применять к каждому разряду устойчивую сортировку подсчётом — от младшего разряда к старшему. После прохода по всем разрядам массив оказывается полностью отсортированным. Сложность алгоритма составляет $O(n \cdot k)$, где k — количество разрядов, а n — число элементов.

2 Исходный код

1 Вспомогательные структуры

Ключом является дата в формате DD.MM.YYYY, которую можно однозначно представить как целое число по формуле $year \times 10000 + month \times 100 + day$. Такое кодирование сохраняет порядок дат при числовом сравнении.

Так как количество входных строк заранее неизвестно, реализован динамический массив `TVector<T>` с удвоением ёмкости при переполнении. Для него реализовано правило пяти: деструктор, конструктор копирования, оператор присваивания копированием, конструктор перемещения и оператор присваивания перемещением.

include/vector.hpp	
<code>template <typename T> class TVector</code>	Динамический массив с правилом пяти; методы <code>PushBack</code> , <code>Size</code> , <code>operator[]</code>
include/pair.hpp	
<code>struct TPair</code>	Пара «ключ-значение»: хранит исходную строку и числовой ключ <code>uint32_t</code>
src/main.cpp	
<code>void CountingPass(TVector<TPair>&, TVector<TPair>&, int)</code>	Один проход сортировки подсчётом по заданному байту ключа
<code>void RadixSort(TVector<TPair>&)</code>	Поразрядная сортировка: четыре прохода по байтам <code>uint32_t</code>

Листинг 1: Структура `TPair` и объявление `TVector`

```
1 struct TPair {
2     std::string raw;
3     uint32_t key;
4
5     explicit TPair(const std::string &line);
6     TPair(const TPair &other);
7     TPair &operator=(const TPair &other);
8     TPair(TPair &&other) noexcept;
9     TPair &operator=(TPair &&other) noexcept;
10 };
11
12 template <typename T>
13 class TVector {
14 public:
15     TVector();
16     explicit TVector(size_t n);
17     TVector(const TVector &other);
18     TVector &operator=(const TVector &other);
```

```

19 |     TVector(TVector &&other) noexcept;
20 |     TVector &operator=(TVector &&other) noexcept;
21 |     ~TVector();
22 |     size_t Size() const;
23 |     void PushBack(T &&value);
24 |     T &operator[](size_t i);
25 |     const T &operator[](size_t i) const;
26 | private:
27 |     T *data;
28 |     size_t size;
29 |     size_t capacity;
30 |     void Reallocate(size_t newCapacity);
31 | };

```

2 Сортировка подсчётом по одному байту

Функция `CountingPass` выполняет один стабильный проход по заданному байту ключа. Сначала строится массив частот (256 элементов — по числу возможных значений байта), затем он преобразуется в массив префиксных сумм. Элементы расставляются в выходной массив в обратном порядке — это обеспечивает устойчивость.

Листинг 2: `CountingPass` — один проход сортировки подсчётом

```

1 | void CountingPass(TVector<TPair> &input, TVector<TPair> &output,
2 |                 int byteIndex) {
3 |     int count[BYTE_BASE];
4 |     for (int i = 0; i < BYTE_BASE; ++i) {
5 |         count[i] = 0;
6 |     }
7 |     for (size_t i = 0; i < input.Size(); ++i) {
8 |         uint8_t byteVal = static_cast<uint8_t>(
9 |             (input[i].key >> (byteIndex * 8)) & BYTE_MASK);
10 |        count[byteVal]++;
11 |    }
12 |    for (int i = 1; i < BYTE_BASE; ++i) {
13 |        count[i] += count[i - 1];
14 |    }
15 |    for (int i = static_cast<int>(input.Size()) - 1; i >= 0; --i) {
16 |        uint8_t byteVal = static_cast<uint8_t>(
17 |            (input[i].key >> (byteIndex * 8)) & BYTE_MASK);
18 |        int pos = --count[byteVal];
19 |        output[pos] = std::move(input[i]);
20 |    }
21 | }

```

3 Поразрядная сортировка

Функция `RadixSort` применяет `CountingPass` последовательно к байтам 0, 1, 2, 3 (от младшего к старшему). Для экономии аллокаций используется заранее выделенный буфер `buffer`: результат каждого прохода перемещается обратно в исходный массив.

Листинг 3: `RadixSort` — четыре прохода по байтам `uint32_t`

```
1 | void RadixSort(TVector<TPair> &data) {  
2 |     if (data.Size() < 2) {  
3 |         return;  
4 |     }  
5 |     TVector<TPair> buffer(data.Size());  
6 |     for (int byte = 0; byte < UINT32_BYTES; ++byte) {  
7 |         CountingPass(data, buffer, byte);  
8 |         for (size_t i = 0; i < data.Size(); ++i) {  
9 |             data[i] = std::move(buffer[i]);  
10 |         }  
11 |     }  
12 | }
```

3 Консоль

```
# cmake -B build -DCMAKE_BUILD_TYPE=Release
# cmake --build build
# ./build/lab1/lab1_main
01.01.2000    Alice
15.06.1995    Bob
03.03.2010    Charlie
20.11.1988    Diana
^D
20.11.1988    Diana
15.06.1995    Bob
01.01.2000    Alice
03.03.2010    Charlie
```

4 Тест производительности

1 Методика

Для сравнения создаются два одинаковых массива из случайно сгенерированных дат. Один сортируется реализованным алгоритмом `RadixSort`, второй — стандартной функцией `std::stable_sort` из STL. Время каждого алгоритма измеряется отдельно с помощью `std::chrono::high_resolution_clock`. Генерация данных и ввод-вывод в замер не входят.

Тест запускается на пяти размерах от 100 до 10^6 элементов.

2 Результаты

N	RadixSort, мкс	std::stable_sort, мкс
100	8	7
1 000	66	74
10 000	831	1 133
100 000	14 291	19 226
1 000 000	140 767	191 842

Таблица 1: Время сортировки в микросекундах

```
# python3 lab1/benchmark/generator.py 100 && ./build/lab1/lab1_benchmark <tests.txt
Radix sort time: 8us
STL stable sort time: 7us
# python3 lab1/benchmark/generator.py 1000 && ./build/lab1/lab1_benchmark <tests.txt
Radix sort time: 66us
STL stable sort time: 74us
# python3 lab1/benchmark/generator.py 10000 && ./build/lab1/lab1_benchmark
<tests.txt
Radix sort time: 831us
STL stable sort time: 1133us
# python3 lab1/benchmark/generator.py 100000 && ./build/lab1/lab1_benchmark
<tests.txt
Radix sort time: 14291us
STL stable sort time: 19226us
# python3 lab1/benchmark/generator.py 1000000 && ./build/lab1/lab1_benchmark
<tests.txt
Radix sort time: 140767us
STL stable sort time: 191842us
```

3 Анализ результатов

На малых объёмах ($N = 100$) оба алгоритма работают за доли миллисекунды и накладные расходы на системный вызов сопоставимы с полезной работой, поэтому разница незначительна.

Начиная с $N = 1\,000$ поразрядная сортировка стабильно опережает `std::stable_sort`. Это соответствует теоретическим оценкам: сложность `RadixSort` составляет $O(n \cdot k)$, где $k = 4$ — фиксированное число байт в `uint32_t`, то есть фактически $O(n)$. Сложность `std::stable_sort` равна $O(n \log n)$.

На $N = 10^6$ поразрядная сортировка работает в **1.36 раза** быстрее (140 767 мкс против 191 842 мкс). Разрыв не такой большой, как ожидался от линейного алгоритма, поскольку `RadixSort` делает 4 полных прохода по массиву (итого 4 прохода по памяти), а `std::stable_sort` использует блочные операции, дружественные кэшу. Тем не менее линейная асимптотика подтверждается: при увеличении N в 10 раз время `RadixSort` растёт примерно в 10 раз, тогда как `std::stable_sort` — чуть быстрее.

5 Дневник выполнения работы

- 05.03.2026: изучено условие задачи, выбран способ кодирования даты в целочисленный ключ и определён формат пары «ключ–строка».
- 05.03.2026: реализованы шаблонный динамический массив, пара значений и стабильная поразрядная сортировка через сортировку подсчётом по байтам.
- 05.03.2026: подготовлен генератор тестов и benchmark, выполнено сравнение с `std::stable_sort` на наборах разного размера.
- 05.03.2026: проверены граничные случаи: пустой ввод, одинаковые ключи, минимальные и максимальные значения даты; оформлен отчёт.

6 Выводы

Выполнив данную лабораторную работу по курсу «Дискретный анализ», я научился реализовывать алгоритм поразрядной сортировки и сортировки подсчётом. Разобрался, как сортировать данные побайтово, используя тип `uint32_t` в качестве ключа, а также как компактно кодировать дату в числовое значение для эффективного сравнения. Вспомнил, как писать шаблонный динамический массив и применять правило пяти.

Список литературы

- [1] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ, 2-е издание*. — Издательский дом «Вильямс», 2007. Перевод с английского: И. В. Красиков, Н. А. Орехова, В. Н. Романов. — 1296 с. (ISBN 5-8459-0857-4 (рус.))
- [2] *Сортировка подсчётом* — *Википедия*.
URL: https://ru.wikipedia.org/wiki/Поразрядная_сортировка (дата обращения: 16.12.2013).
- [3]